

Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО

Факультет безопасности информационных технологий

Дисциплина:
«Информационная безопасность баз данных»

ОТЧЕТ ПО ЛАБОРАТОРНОЙ РАБОТЕ №3
«Функции и триггеры в БД»

Выполнил:

Пахомов Владимир Юрьевич, студент группы N3250



(подпись)

Проверил:

Ярцева Наталия Андреевна, преподаватель ФБИТ

(отметка о выполнении)

(подпись)

Санкт-Петербург

2026г.

СОДЕРЖАНИЕ

СОДЕРЖАНИЕ.....	2
ВВЕДЕНИЕ.....	3
ХОД РАБОТЫ.....	3
Задание 1.....	4
Задание 2.....	5
Задание 3.....	6
Задание 4.....	8
Задание 5.....	9
ЗАКЛЮЧЕНИЕ.....	10

ВВЕДЕНИЕ

Цель работы — получение навыков написания процедур, функций и триггеров в БД.

ХОД РАБОТЫ

В данной лабораторной работе для взаимодействия с базой данных PostgreSQL будем использовать утилиту psql на ОС Linux. Для создания и подключения к локальной базе данных создадим файл lab3.sql и впишем в него все команды для создания базы данных и отправки команд из методического материала. Для подключения к базе данных используем команду:

```
psql -U postgres -f lab3.sql
```

Для решения задач была создана база данных с информацией об оперативной памяти и её продажах:

```
-- Таблица товаров
CREATE TABLE rams (
  id SERIAL PRIMARY KEY,
  name TEXT,
  standard VARCHAR(10),
  price NUMERIC,
  total_sales_sum NUMERIC DEFAULT 0
);

-- Таблица продаж
CREATE TABLE sales (
  id SERIAL PRIMARY KEY,
  ram_id INT REFERENCES rams(id),
  amount NUMERIC,
  sale_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

Задание 1

Создадим функцию, которая будет пересчитывать сумму продаж для всех лотов оперативной памяти и будет обновлять поле total_sales_sum в таблице products:

```
CREATE OR REPLACE FUNCTION refresh_total_sales() RETURNS void AS $$
BEGIN
    UPDATE rams AS p
    SET total_sales_sum = (
        SELECT SUM(amount)
        FROM sales AS s WHERE p.id = s.ram_id
    );
END;
$$ LANGUAGE plpgsql;
```

Для примера вот вывод таблицы с данными таблицы rams до и после вызова функции refresh_total_sales:

id	name	standard	price	total_sales_sum
1	ADATA	ddr3	22000	0
2	HYPERX	ddr4	45000	0
3	CRUCIAL	ddr5	70000	0

(3 rows)

refresh_total_sales

(1 row)

id	name	standard	price	total_sales_sum
1	ADATA	ddr3	22000	32000
2	HYPERX	ddr4	45000	1000
3	CRUCIAL	ddr5	70000	7000

(3 rows)

Задание 2

Создадим триггер, который будет реагировать на вставку новых строк в таблицу sales и будет выполнять то же действие, что в задании 1:

```
CREATE OR REPLACE FUNCTION trigger_update_stock() RETURNS trigger AS $$
BEGIN
    UPDATE rams SET total_sales_sum = total_sales_sum + NEW.amount
    WHERE id = NEW.ram_id;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trigger_after_insert_sale AFTER INSERT
ON sales FOR EACH ROW EXECUTE PROCEDURE trigger_update_stock();
```

Пример работы триггера при вставке новой продажи в таблицу sales:

id	name	standard	price	total_sales_sum
1	ADATA	ddr3	22000	32000
2	HYPERX	ddr4	45000	1000
3	CRUCIAL	ddr5	70000	4000

(3 rows)

INSERT 0 1

id	name	standard	price	total_sales_sum
1	ADATA	ddr3	22000	32000
2	HYPERX	ddr4	45000	1000
3	CRUCIAL	ddr5	70000	7000

(3 rows)

Задание 3

Создадим триггер, который будет добавлять данные с продаж в зависимости от года. Продажи 2024 и 2025 года будут отправляться в разные таблицы (sales_2024 и sales_2025), которые будут наследовать родительскую sales:

```
CREATE TABLE sales_2024 (CHECK (sale_date >= '2024-01-01' AND sale_date < '2025-01-01')) INHERITS (sales);
CREATE TABLE sales_2025 (CHECK (sale_date >= '2025-01-01' AND sale_date < '2026-01-01')) INHERITS (sales);

CREATE OR REPLACE FUNCTION trigger_route_by_year() RETURNS trigger AS $$
BEGIN
    IF (NEW.sale_date >= '2024-01-01' AND NEW.sale_date < '2025-01-01') THEN
        INSERT INTO sales_2024 VALUES (NEW.*);
    ELSIF (NEW.sale_date >= '2025-01-01' AND NEW.sale_date < '2026-01-01') THEN
        INSERT INTO sales_2025 VALUES (NEW.*);
    ELSE
        RAISE EXCEPTION 'Дата вне поддерживаемого диапазона';
    END IF;
    RETURN NULL;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trigger_sales_partitioning
BEFORE INSERT ON sales
FOR EACH ROW EXECUTE PROCEDURE trigger_route_by_year();
```

Пример работы триггера при добавлении продажи 2025 года:

```
sales
 id | ram_id | amount | sale_date
----+-----+-----+-----
  1 |      1 |  22000 | 2024-10-10 10:00:00
  2 |      1 |  10000 | 2024-11-12 12:00:00
  3 |      2 |   1000 | 2024-11-12 12:00:00
  4 |      3 |   4000 | 2024-11-12 12:00:00
(4 rows)
```

```
INSERT 0 0
```

```
sales
 id | ram_id | amount | sale_date
----+-----+-----+-----
  1 |      1 |  22000 | 2024-10-10 10:00:00
  2 |      1 |  10000 | 2024-11-12 12:00:00
  3 |      2 |   1000 | 2024-11-12 12:00:00
  4 |      3 |   4000 | 2024-11-12 12:00:00
(4 rows)
```

```
sales_2024
 id | ram_id | amount | sale_date
----+-----+-----+-----
(0 rows)
```

sales_2025

id	ram_id	amount	sale_date
5	3	3000	2025-02-02 00:00:00

(1 row)

Задание 4

Создадим триггер, который будет реагировать на вставку данных в таблицу rams и проверять стандарт памяти. Если добавляемого стандарта нет в таблице ram_standards, то значение standard у нового поля будет подменяться на 'undefined':

```
CREATE TABLE ram_standards (  
  id SERIAL PRIMARY KEY,  
  name VARCHAR(5)  
);  
INSERT INTO ram_standards (name) VALUES  
( 'ddr3' ),  
( 'ddr4' ),  
( 'ddr5' );  
  
CREATE OR REPLACE FUNCTION trigger_check_standard() RETURNS trigger AS $$  
BEGIN  
  IF NEW.standard NOT IN (SELECT name FROM ram_standards) THEN  
    NEW.standard := 'undefined';  
  END IF;  
  RETURN NEW;  
END;  
$$ LANGUAGE plpgsql;  
  
CREATE TRIGGER trigger_standard  
BEFORE INSERT ON rams  
FOR EACH ROW EXECUTE PROCEDURE trigger_check_standard();
```

Пример работы триггера при добавлении несуществующего стандарта памяти:

```
INSERT 0 1  
id | name      | standard | price | total_sales_sum  
----+-----+-----+-----+-----  
1  | ADATA     | ddr3     | 22000 | 0  
2  | HYPERX    | ddr4     | 45000 | 0  
3  | CRUCIAL   | ddr5     | 70000 | 0  
4  | ATERMITER | undefined | 1000  | 0  
(4 rows)
```


Задание 5

Создадим триггер, который будет реагировать на события INSERT, UPDATE и DELETE с таблицей rams и в зависимости от события будет записывать в таблицу audit_log разные логи, соответствующие своему событию:

```
CREATE TABLE audit_log (  
    id SERIAL PRIMARY KEY,  
    table_name TEXT,  
    operation TEXT,  
    old_data JSONB,  
    new_data JSONB,  
    changed_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);  
  
CREATE OR REPLACE FUNCTION trigger_audit_log() RETURNS trigger AS $$  
BEGIN  
    IF TG_OP = 'DELETE' THEN  
        INSERT INTO audit_log (table_name, operation, old_data)  
        VALUES (TG_TABLE_NAME, TG_OP, to_jsonb(OLD));  
        RETURN OLD;  
    ELSIF TG_OP = 'UPDATE' THEN  
        INSERT INTO audit_log (table_name, operation, old_data, new_data)  
        VALUES (TG_TABLE_NAME, TG_OP, to_jsonb(OLD), to_jsonb(NEW));  
        RETURN NEW;  
    ELSIF TG_OP = 'INSERT' THEN  
        INSERT INTO audit_log (table_name, operation, new_data)  
        VALUES (TG_TABLE_NAME, TG_OP, to_jsonb(NEW));  
        RETURN NEW;  
    END IF;  
    RETURN NULL;  
END;  
$$ LANGUAGE plpgsql;  
  
CREATE TRIGGER trigger_audit_rams  
AFTER INSERT OR UPDATE OR DELETE ON rams  
FOR EACH ROW EXECUTE PROCEDURE trigger_audit_log();
```

Пример работы триггера при событиях DELETE, UPDATE и INSERT:

id	table_name	operation	old_data	new_data	changed_at
1	rams	UPDATE	{"id": 3, "name": "CRUCIAL", "price": 70000, "standard": "ddr5", "total_sales_sum": 0}	{"id": 3, "name": "CRUCIAL", "price": 1000, "standard": "ddr5", "total_sales_sum": 0}	2026-03-03 22:56:45.211027
2	rams	INSERT		{"id": 4, "name": "ATERMITER", "price": 1000, "standard": "ddr1", "total_sales_sum": 0}	2026-03-03 22:56:45.212355
3	rams	DELETE	{"id": 1, "name": "ADATA", "price": 22000, "standard": "ddr3", "total_sales_sum": 0}		2026-03-03 22:56:45.213725

(3 rows)

ЗАКЛЮЧЕНИЕ

В ходе лабораторной работы были изучены механизмы PL/pgSQL для создания функций и триггеров в СУБД PostgreSQL. Основными результатами можно выделить:

- Автоматизацию: Реализована процедура и триггер для автоматической агрегации данных, что гарантирует их согласованность без участия пользователя.
- Управление данными: Настроен триггер для распределения записей по таблицам на основе дат, что оптимизирует хранение информации.
- Валидацию и нормализацию: С помощью триггеров типа BEFORE реализована автоматическая подмена значений по словарю, минимизирующая ошибки ручного ввода.
- Безопасность и аудит: На базе специальных переменных (NEW, OLD, TG_OP) создана система логирования изменений. Это обеспечивает прослеживаемость действий, необходимую для защиты информации.

По итогу можно выделить, что использование триггеров позволяет перенести критическую бизнес-логику и контроль целостности на уровень СУБД, обеспечивая надёжность данных независимо от внешних приложений.